Sarabo — Viva / Q&A Practice

নিজের প্র্যাকটিসের জন্য। Presentation English-এ, Q&A বাংলা/মিক্সে। সব file path আসল code থেকে মিলিয়ে লেখা (server: `sarabo-server/`, client: `sarabo-client/src/`)। ১৮ অক্টোবর ২০২৬।

উত্তর দেওয়ার নিয়ম: (১) আগে এক লাইনে মূল কথা, (২) তারপর "কোথায় আছে" — client file → API → server route → controller → service, (৩) একটা edge case বলে শেষ। না জানলে বলো: "এটা এখন নেই, limitation-এ লেখা আছে, future work-এ রেখেছি" — বানিয়ে বলবে না।

১. মুখস্থ রাখার মতো সংখ্যা (Quick facts)

বিষয়	মান	কোথায় (code)
Roles	Customer, Technician, Admin (DB-তে `user`, `rider`, `admin`)	`middleware/auth.js`
Request states	14টা (pending-pickup … repair_completed, parcel_delivered, cancelled)	`utils/repairRequestStatus.js`
Services	45টা service, 8 ধরনের device	`service_definitions` collection
Commission	Technician 90%, Sarabo 10% — পুরো quote total-এর উপর, পূর্ণ টাকায়	`utils/settlement.js`
Minimum inspection fee	100 BDT; estimate-এর max 1,000-এর বেশি হলে 200	`utils/inspectionFee.js` → `minimumInspectionFee()`
Pickup slots	দিনে ৪টা ২-ঘণ্টার window (10:00–18:00), শুক্রবার বন্ধ, ৭ দিন আগে পর্যন্ত, কমপক্ষে ১ ঘণ্টা আগে, প্রতি region-এ slot প্রতি ৩ জন	`utils/pickupSlots.js`
Offer ranking weights	Rating 30%, reliability 25%, price 20%, district 15%, reply time 10%	`services/offerRanking.js`
Warranty	30 দিন; প্রতি repair-এ max 2 claim, একসাথে 1টা খোলা; book 3 দিনে, decide 3 দিনে, re-repair 7 দিনে; photo max 4	`utils/warranty.js`
Auto deadlines	Receipt auto-confirm 3 দিন; unpaid approved quote close 3 দিন; declined quote auto close 3 দিন	`services/receiptConfirmation.js`, `requestClosure.js`, `declinedQuoteClose.js`
Cancel booking	Pickup-এর ≥2 ঘণ্টা আগে cancel = fee ফেরত; পরে = fee রাখা হয় (90/10)	`controllers/inspectionFeeController.js`
Damage photos	Max 3টা, JPEG/PNG/WebP, 8 MB; private bucket, short-lived signed URL	`utils/damageUpload.js`, `services/damageStorageService.js`
Tracking code	128-bit random (16 bytes, base64url), যেমন `SRB-eYVvd7…`	`utils/trackingId.js` → `generateSecureTrackingId()`
Turned-away demand	Customer email hash করে, 180 দিন রাখা হয়	`services/technicianMatchGate.js`
Cron	প্রতিদিন 12:00 UTC (বাংলাদেশ 18:00), `GET /cron/pickup-reminders`	`vercel.json`, `controllers/pickupReminderController.js`

Notification types	47টা	`utils/notificationEvents.js`
API	125টা route, 25টা route file	`routes/*.js`
Tests	2,761 server test (real HTTP + real MongoDB replica set), 58 Playwright browser test, CI প্রতিটা push-এ	`tests/`, `test-all.js`, `.github/workflows`
Hosting	Client ও API দুটোই Vercel-এ; API Mumbai region (`bom1`), MongoDB Atlas-এর পাশে	`vercel.json`
AI	Google Gemini: `gemini-3.5-flash-lite`, fallback `gemini-3.1-flash-lite`	`services/aiProviders/`

২. Code-এর গঠন — একটা request কীভাবে চলে

যেকোনো feature-এর "code কোথায়" প্রশ্নে এই chain-টা বলবে:

```
Browser (React page / component)
  → hook (TanStack Query, যেমন hooks/useJobPortal.js)
  → api/*.js function (axios call)
  → hooks/useAxiosSecure.jsx  (Firebase ID token → "Authorization: Bearer ...")
  → Express route  (server routes/*.js)
  → middleware: verifyFBToken → ensureDatabaseReady → verifyAdmin / verifyTechnician /
verifyEmailVerified
  → controller method  (controllers/*Controller.js — role + state check)
  → service / utils  (services/*.js — business rule, transaction)
  → MongoDB (collections) / Stripe / Supabase / Gemini / Gmail
```

Server folder (`sarabo-server/`)

Folder	কী থাকে
`index.js`	Express app তৈরি, CORS, middleware, সব route register
`routes/`	শুধু URL → middleware → controller method mapping (25 file)
`middleware/`	`auth.js` (token, role, email verified), `database.js`, `requestId.js`, `logging.js`, `supportRateLimit.js`, `mongoRateLimitStore.js`
`controllers/`	প্রতিটা domain-এর state rule; `controllers/index.js` সব controller বানিয়ে key দেয় (`repairRequest`, `jobPortal`, `quote` ...)
`services/`	Shared logic: booking, refund, settlement, matching, AI, email, notification
`utils/`	Pure rule/validation: status list, quote, settlement হিসাব, pickup slot, warranty, tracking id
`config/`	Database, Firebase Admin, Stripe, Supabase, AI, email, CORS config
`models/`, `data/`, `scripts/`	Model helper, seed data (catalogue), migration/seed script (dry-run আগে)
`tests/`	Integration test suites (fake Stripe/AI/storage)

Client folder (`sarabo-client/src/`)

Folder	কী থাকে

`routes/router.jsx`	সব page route; guard: `PrivateRoute`, `CustomerRoute`, `TechnicianRoute`, `AdminRoute`
`pages/`	Public page (Home, Services, ServiceAreas, Jobs, TrackRequest) আর `pages/Dashboard/*`
`components/`	Feature অনুযায়ী: `repair-request`, `technician`, `customer`, `quote`, `inspection`, `repair`, `warranty`, `workflow`, `spine`, `admin`, `support` ...
`api/`	প্রতিটা server endpoint-এর axios function (19 file)
`hooks/`	Data hook (useQuery/useMutation), `useAuth`, `useRole`, `useAxiosSecure`
`contexts/AuthContext`, `firebase/firebase.init.js`	Firebase login state
`config/`, `utils/`	Navigation, status label, presentation helper

৩. Feature অনুযায়ী Code Map

Format: Client file → API → Server (route → controller → service/utils) → মূল logic।

৩.১ Login, role আর route guard

```
Client: pages/Auth/Login/Login.jsx, Register/Register.jsx, SocialLogin/SocialLogin.jsx (Google)
        contexts/AuthContext + firebase/firebase.init.js
        hooks/useRole.jsx → GET /users/me/role
        routes/AdminRoute.jsx, TechnicianRoute.jsx, CustomerRoute.jsx
API:    POST /users (প্রথম login-এ user তৈরি), GET /users/me/role
Server: routes/users.js → controllers/userController.js
        middleware/auth.js → verifyFBToken (admin.auth().verifyIdToken), verifyAdmin,
verifyTechnician, verifyEmailVerified
```

Logic: Login Firebase করে; প্রতিটা call-এ `useAxiosSecure.jsx` interceptor `getIdToken()` দিয়ে token লাগায়, 401 হলে token refresh করে একবার retry। Server token verify করে email বের করে, DB থেকে role দেখে। Role ঠিক না হলে 403। Client guard শুধু দেখানোর জন্য — আসল নিরাপত্তা server-এ।

৩.২ Repair request তৈরি (region gate, pickup slot)

```
Client: components/repair-request/RepairRequestV2Form.jsx (form)
        components/repair-request/ServiceDefinitionSelector.jsx, EstimateCard.jsx
        components/pickup/PickupSlotPicker.jsx  (hooks/usePickupSlots.js → GET /pickup-slots)
        hooks/useServiceAvailability.js → GET /service-availability (submit-এর আগে warning)
        api/repairRequests.js → createRepairRequestV2()
API:    POST /repair-requests
Server: routes/repairRequests.js → repairRequestController.createRepairRequest() →
createRepairRequestV2()
        services/technicianMatchGate.js → hasTechnicianFor(), recordTurnedAway()
        utils/pickupSlots.js + pickupSlotController.checkSlot()
        utils/trackingId.js → generateSecureTrackingId()
```

Logic: Validate → service definition load → region-এ ওই device+repair+level-এর approved technician আছে কি না দেখে, না থাকলে `409 NO_TECHNICIAN_AVAILABLE` আর turned-away count (hash করা) → slot ভরা হলে `409`

`PICKUP_SLOT_FULL` → price snapshot রেখে document save, tracking code, notification + email, আর job board-এ post (`services/jobAnnouncements.js`)। Request তৈরিতে টাকা লাগে না।

৩.৩ AI request helper (Banglish → form)

```
Client: components/repair-request/AiRequestHelper.jsx, api/aiHelper.js, hooks/useAiStatus.js
API:    GET /ai/status, POST /ai/request-draft
Server: routes/aiRequestDraft.js → controllers/aiRequestDraftController.js
        services/aiRequestDraftService.js (buildCatalogue, validateDraft)
        services/aiProviders/geminiRequestDraftProvider.js, services/aiHelperCall.js (timeout)
```

Logic: Live catalogue-এর id list Gemini-কে দেওয়া হয়; উত্তরে যে service id আসে সেটা catalogue-এ না থাকলে বাতিল (`validateDraft`)। AI দাম ঠিক করে না — দাম catalogue থেকে। Customer সব বদলাতে পারে, Undo আছে। Switch: `AI_REQUEST_HELPER_ENABLED`।

৩.৪ Damage photo upload

```
Client: components/damage-images/DamageImageManager.jsx, DamageImagePicker.jsx,
PostCreationDamageStep.jsx
        api/damageImages.js, hooks/useDamageImages.js, useDamageImageMutations.js
API:    POST /repair-requests/:id/damage-images/upload-session
        (browser সরাসরি Supabase-এ PUT করে signed URL দিয়ে)
        POST /repair-requests/:id/damage-images/finalize
        GET  /repair-requests/:id/damage-images,  DELETE …/:imageId
Server: routes/damageUploads.js → controllers/damageUploadController.js
        services/damageStorageService.js, services/supabaseBucketAdapter.js, utils/damageUpload.js
```

Logic: Server আগে type/size check করে upload session দেয় (20 মিনিট)। Finalize-এ storage থেকে আসল object-এর type আর size আবার verify করে (`storage.verifyObject`), তারপর transaction-এ attach (max 3)। দেখার সময় শুধু owner, admin আর booked technician কয়েক মিনিটের signed URL পায়।

৩.৫ Job board আর offer পাঠানো

```
Client: pages/Jobs/Jobs.jsx, components/technician/JobCard.jsx, TechnicianJobFilters.jsx
        components/technician/JobApplicationSheet.jsx (offer form)
        hooks/useJobPortal.js, api/jobPortal.js
API:    GET /jobs (public, safe summary)            GET /technician/job-portal (technician)
        GET /repair-requests/:id/applications/price-guide
        POST /repair-requests/:id/applications/ai-note
        POST /repair-requests/:id/applications      POST …/applications/mine/withdraw
Server: routes/jobPortal.js → controllers/jobPortalController.js
        utils/jobPortal.js (offer validation, fee range), services/offerPriceGuide.js,
        services/aiOfferNoteService.js (removeContactDetails), services/assignmentEligibility.js
```

Logic: Public board-এ customer-এর নাম/ঠিকানা নেই। Technician শুধু নিজের region-এর, নিজের skill-এর job-এ offer দিতে পারে। Fee minimum থেকে catalogue fee-র মধ্যে হতে হবে। Price guide = একই repair-এর recent offer-এর মাঝের অর্ধেক; ৩টার কম হলে catalogue দাম। AI note থেকে phone number (বাংলা সংখ্যাসহ), email, link সরানো হয়।

৩.৬ Offer ranking (Smarter matching)

```
Client: components/customer/TechnicianApplications.jsx, components/technician/OfferCard.jsx
API:    GET /repair-requests/:id/applications
Server: jobPortalController.listForRequest() → services/offerRanking.js → rankOffers()
```

Logic: ৫টা factor 0–1 score: rating (3 star-এর দিকে smooth, যাতে ১টা 5-star review ৪০টা 4.8-কে না ছাড়ায়), reliability `(completed+1)/(completed+noShows+2)`, price (অন্য offer-এর তুলনায়), district (একই হলে 1, নাহলে 0.4), reply time (median)। নতুন technician-এর neutral 0.5 — শাস্তিও না, বোনাসও না। Reason tag তখনই দেখায় যখন তথ্য যথেষ্ট শক্ত। Customer নিজেই বেছে নেয়।

৩.৭ Offer choose + inspection fee payment + booking

```
Client: TechnicianApplications.jsx → "Choose" → ConfirmDialog → api/jobPortal.js acceptApplication()
API:    POST /repair-requests/:id/applications/:applicationId/accept  → Stripe Checkout URL
        POST /stripe-webhook  (Stripe → server)
Server: jobPortalController.accept() → services/checkoutSessionManager.js (একটাই খোলা checkout)
        paymentController.handleStripeWebhook() → stripe.webhooks.constructEvent (signature)
        services/paymentProcessor.js → services/jobBooking.js → bookApplication() (transaction)
        services/inspectionFeeRefunds.js (book না হলে refund)
```

Logic: `bookApplication` একটা MongoDB transaction-এ: offer এখনো `pending`? technician অন্য active job-এ busy না? request এখনো `pending-pickup`? → request `driver_assigned`, technician `in_delivery`, এই offer `accepted`, বাকি সব `not_chosen`। কোনো শর্ত ভাঙলে কিছুই হয় না, fee নিজে থেকে refund (idempotency key `inspection-refund-<sessionId>`)।

৩.৮ Pickup, condition record, inspection

```
Client: pages/Dashboard/AssignedJobs/AssignedJobs.jsx, hooks/useTechnicianJobs.js
        components/technician/TechnicianRequestDetailsView.jsx
        components/inspection/InspectionForm.jsx, DetectedIssuesEditor.jsx
API:    PATCH /repair-requests/:id/status  (Start pickup → rider_arriving; Device received →
parcel_picked_up + pickupCondition)
        POST /repair-requests/:id/inspection
        POST /repair-requests/:id/pickup-condition/dispute (customer), …/customer-not-available
(technician)
Server: repairRequestController.updateRepairRequestStatus() (utils/repairRequestStatus.js →
isValidTransition)
        controllers/inspectionController.js + utils/inspection.js
        controllers/workflowController.js
```

Logic: Generic status PATCH শুধু ছোট কয়েকটা transition দেয় (`ALLOWED_TRANSITIONS`); inspection, quote, payment-এর state শুধু নিজের endpoint দিয়ে বদলায়। Inspection-এ `not_repairable` / `not_economical` হলে যেকোনো পক্ষ request close করতে পারে।

৩.৯ Quote, approve আর repair payment

```
Client: components/quote/QuoteForm.jsx (technician), QuoteSection.jsx, QuoteDecisionActions.jsx
(customer)
        components/payment/V2PaymentSection.jsx, api/quotes.js, api/payments.js
        pages/Dashboard/Payment/PaymentSuccess.jsx, components/payments/ReceiptDownloadButton.jsx
API:    POST /repair-requests/:id/quote,  POST …/quote/decision,  POST …/quote/revise
        POST /repair-requests/:id/checkout-session → Stripe → POST /stripe-webhook
        GET /payments/:transactionId/receipt.pdf
Server: controllers/quoteController.js + utils/quote.js (validateQuoteSubmission)
        controllers/paymentController.js, services/paymentEligibility.js,
services/paymentProcessor.js
        services/paymentReceipt.js (pdfkit)
```

Logic: Quote = labour + parts + additional; total server হিসাব করে। Approve করলে বাকি = total − আগে দেওয়া fee (4,000 − 200 = 3,800)। Webhook-এ payment record হয় একবারই; একই transaction-এ settlement snapshot freeze (`buildSettlementDocument`)। Quote decline হলে একই technician revise করে নতুন quote দিতে পারে (`POST …/quote/revise`, পুরনোটা archive হয়), নাহলে close।

৩.১০ Repair, return, receipt confirm

```
Client: components/repair/RepairProgressForm.jsx, RepairCompletionForm.jsx,
ReceiptConfirmationSection.jsx
        components/workflow/RequestWorkflowActions.jsx (mark-returned, receipt-dispute, cannot-
complete …)
API:    POST /repair-requests/:id/repair/start | progress | complete
        POST /repair-requests/:id/mark-returned (technician)
        POST /repair-requests/:id/confirm-receipt (customer)   POST …/receipt-dispute
Server: controllers/repairController.js, controllers/workflowController.js
        services/receiptConfirmation.js → confirmReceiptOnce()
```

Logic: Customer confirm করলে (বা ৩ দিন পর cron auto-confirm, বা admin) settlement `pending → available`, warranty শুরু। `confirmReceiptOnce` একবারই কাজ করে — দুইবার চাপলেও double হয় না।

৩.১১ Wallet আর bKash withdrawal

```
Client: pages/Dashboard/Wallet/Wallet.jsx, hooks/useTechnicianWallet.js
        pages/Dashboard/WithdrawalRequests/WithdrawalRequests.jsx (admin, Mark paid dialog)
API:    GET /technician/wallet, POST /technician/withdrawals
        GET /admin/withdrawals, POST /admin/withdrawals/:id/mark-paid | reject
Server: routes/wallet.js → controllers/walletController.js → utils/settlement.js → calculateWallet()
```

Logic: Balance DB-তে রাখা হয় না — প্রতিবার settlement আর withdrawal থেকে হিসাব হয়, তাই কখনো ভুল জমে না। একসাথে একটাই খোলা withdrawal (unique partial index)। Mark paid-এ bKash TrxID লাগে (8–12 অক্ষর), একই TrxID আবার দিলে `409 BKASH_TRX_ID_ALREADY_USED`।

৩.১২ Warranty

```
Client: components/warranty/CustomerWarrantySection.jsx, WarrantyClaimCard.jsx
        pages/Dashboard/WarrantyClaims/WarrantyClaims.jsx (technician + admin), api/warranty.js
API:    POST /repair-requests/:id/warranty-claims (customer)
        POST /warranty-claims/:claimId/schedule | decision | rerepair (technician)
        POST /warranty-claims/:claimId/confirm (customer "Working now")
        POST /admin/warranty-claims/:claimId/review | decision | cannot-fix | refund
Server: routes/warranty.js → controllers/warrantyController.js
        utils/warranty.js (window, deadline, validation), services/warrantyRefunds.js,
services/refundParts.js
```

Logic: প্রতিটা step একটা guarded update — আগের step না হলে পরেরটা হয় না, দুইজন একসাথে চাপলে একজনই সফল। Deadline মিস হলে claim overdue, admin take over। Refund = repair payment + fee, Stripe-এ প্রতিটা অংশের নিজস্ব idempotency key; settlement `reversed`, আগে তোলা টাকা পরের আয় থেকে কাটা হয়।

৩.১৩ Daily cron (reminder + deadline)

```
vercel.json → "crons": [{ "path": "/cron/pickup-reminders", "schedule": "0 12 * * *" }]
Server: routes/pickupSlots.js → controllers/pickupReminderController.js → sendReminders()
        secret check (CRON_SECRET), remindInApp() + email
        runWorkflowJobs(): closeUnpaidApprovedQuotes, receipt auto-confirm,
                           warranty.alertOverdueClaims(), workflow.alertStalled()
        closeStaleDeclinedRequests() (services/declinedQuoteClose.js)
```

Logic: প্রতিটা job আলাদা `attempt()` -এ চলে — একটা fail করলে বাকিগুলো থামে না। Reminder এক request + slot-এ একবারই (de-duplication key)। Live site-এ email + cron দুটোই চালু।

৩.১৪ Public tracking

```
Client: pages/TrackRequest/TrackRequest.jsx → GET /public/trackings/:trackingCode
Server: routes/publicTracking.js → controllers/trackingController.js → getPublicTracking()
```

Logic: শুধু stage, status আর সময় দেখায়; নাম, email, ঠিকানা, payment কিছুই না। Code 128-bit random, তাই অনুমান করা কার্যত অসম্ভব।

৩.১৫ AI support chat

```
Client: components/support/SupportWidget.jsx, SupportSheet.jsx, hooks/useSupportChat.js,
api/support.js
API:    POST /support/chat  (login ছাড়াও; rate limit)
Server: routes/support.js → controllers/supportController.js → services/aiSupportService.js
        services/supportKnowledgeService.js (live catalogue থেকে prompt)
        services/aiProviders/geminiSupportProvider.js (fallback model)
        middleware/supportRateLimit.js + middleware/mongoRateLimitStore.js
```

Logic: Prompt-এ live catalogue থাকে, তাই দাম সবসময় এখনকার। Follow-up button fixed list থেকে (`utils/supportActions.js`)। Signed-out user 10 মিনিটে 8টা প্রশ্ন; limit MongoDB-তে রাখা, তাই serverless instance বদলালেও কাজ করে।

৩.১৬ Admin attention, reports, AI summary

```
Client: components/admin/AdminOverview.jsx, hooks/useAdminAttention.js → GET /admin/attention
        pages/Dashboard/Reports/Reports.jsx, components/admin/reports/RequestsTrendChart.jsx,
        components/admin/AiReportSummary.jsx → GET/POST /admin/reports/ai-summary
Server: controllers/adminAttentionController.js
        controllers/adminReportController.js → buildReport(days)  (overview, daily, regions,
breakdowns, workload, inspectionFees)
        controllers/adminReportSummaryController.js → services/aiReportSummaryService.js
(onlyKnownNumbers)
```

Logic: AI summary শুধু aggregate সংখ্যা পায়; উত্তরের কোনো sentence-এ এমন সংখ্যা থাকলে যা data-তে নেই, সেই sentence বাদ (`onlyKnownNumbers`)। শেষ summary period অনুযায়ী save থাকে (`ai_report_summaries`)।

৩.১৭ Catalogue, technician approval, users

```
Catalogue: pages/Dashboard/ServiceCatalogue/ServiceCatalogue.jsx, components/admin/ServiceEditor.jsx
           → /admin/service-definitions → controllers/adminServiceDefinitionController.js
(pricingRule.version++)
Technician: pages/BecomeTechnician/BecomeTechnician.jsx → POST /technicians
           pages/Dashboard/ApproveTechnicians/ApproveTechnicians.jsx → PATCH /technicians/:id,
           GET /technicians/:id/final-payout → services/technicianRemovalPayout.js
           Profile: components/technician/TechnicianDetailsEditor.jsx → PATCH /technicians/me
Users:     pages/Dashboard/UsersManagement/UsersManagement.jsx → PATCH /users/:id/role (admin only)
Reviews:   components/feedback/customer/ReviewDialog.jsx → POST /repair-requests/:id/review
           components/feedback/admin/ModerationDialog.jsx → PATCH /admin/technician-
reviews/:id/visibility
           → controllers/feedbackController.js, utils/feedback.js
```

৩.১৮ Service spine (৪ ধাপের progress bar) আর request page

```
pages/Dashboard/RequestDetails/RequestDetails.jsx → role দেখে
   CustomerRequestDetailsView / TechnicianRequestDetailsView / AdminRequestDetailsView
components/spine/ServiceSpine.jsx (Request → Inspect → Approve → Repaired)
components/workspace/NextStepPanel.jsx ("Your next step")
```

৪. "এই code কোথায়?" — ধাপে ধাপে বলার practice

প্রশ্ন: Customer "Create request" চাপলে কী কী হয়? File ধরে বলো।

> **উত্তর:** `RepairRequestV2Form.jsx` form validate করে `api/repairRequests.js`-এর `createRepairRequestV2()` ডাকে → `POST /repair-requests`। `useAxiosSecure` token লাগায়। Server-এ `routes/repairRequests.js`: `verifyFBToken`, `verifyEmailVerified` → `repairRequestController.createRepairRequest()` → v2 হলে `createRepairRequestV2()`। সেখানে `hasTechnicianFor()` (region gate), `checkSlot()` (slot capacity), price snapshot, `generateSecureTrackingId()`, insert, তারপর notification/email/job announcement। Response-এ request id আর tracking code; client "Request created" দেখায় (photo add করার option)।

প্রশ্ন: Technician offer পাঠানোর API call কোথা থেকে যায়?

> **উত্তর:** `components/technician/JobApplicationSheet.jsx` → `api/jobPortal.js` → `POST /repair-requests/:id/applications` → `routes/jobPortal.js` (`verifyFBToken`, `verifyTechnician`) → `jobPortalController.apply()`। Validation `utils/jobPortal.js`-এ (estimate range, fee minimum থেকে catalogue fee)।

প্রশ্ন: Payment "paid" কোথায় হয়? Success page-এ?

> **উত্তর:** না। Success page শুধু দেখায়। আসল "paid" হয় `POST /stripe-webhook`-এ: `routes/paymentWebhook.js` raw body নেয়, `paymentController.handleStripeWebhook()` `stripe.webhooks.constructEvent()` দিয়ে signature verify করে, তারপর `services/paymentProcessor.js` transaction-এ payment row, request status আর settlement লেখে। একই event দুইবার এলে দ্বিতীয়বার কিছু হয় না (idempotent)।

প্রশ্ন: Commission হিসাবের code দেখাও।

> **উত্তর:** `utils/settlement.js`। Base = `partsAmount + laborAmount + additionalCharges` (= quote total)। Commission = integer ratio 10/100, আর technician-এর অংশ = subtotal − commission (বিয়োগ করে, আলাদা % নয়), তাই সবসময় `commission + receivable = subtotal`। Parts/labour কীভাবে ভাগ করল তাতে কারো টাকা বদলায় না।

প্রশ্ন: Double booking কীভাবে আটকানো হয়েছে, code-এ কোথায়?

> **উত্তর:** `services/jobBooking.js` → `bookApplication()`। Transaction-এর ভেতরে technician-এর অন্য active request আছে কি না দেখে (`ACTIVE_STATUSES`), request update-এর filter-এ `deliveryStatus: 'pending-pickup'` আর `technicianId` নেই এমন শর্ত, technician update-এ `workStatus: 'available'` শর্ত। দুইজন একসাথে চাপলে একজনের update match করে না → সব rollback।

প্রশ্ন: Role check কোথায়? Client-এ লুকিয়ে রাখলেই কি যথেষ্ট?

> **উত্তর:** না। Client guard (`routes/AdminRoute.jsx` ইত্যাদি) শুধু UI। Server-এ `middleware/auth.js`: `verifyAdmin` (role !== 'admin' → 403), `verifyTechnician` (role !== 'rider' → 403)। তার উপর controller দেখে এই request-এর owner/assigned technician কি না, আর বর্তমান state-এ এই কাজ চলে কি না।

প্রশ্ন: Pickup slot-এর নিয়ম কোথায়?

> **উত্তর:** `utils/pickupSlots.js`: `PICKUP_SLOTS` (৪টা), `CLOSED_WEEKDAYS = [5]` (শুক্রবার), `BOOKING_WINDOW_DAYS = 7`, `MIN_LEAD_MINUTES = 60`, `DEFAULT_SLOT_CAPACITY = 3`, সময় `Asia/Dhaka`। Availability দেখায় `GET /pickup-slots` (`pickupSlotController`), reschedule `PATCH /repair-requests/:id/pickup-slot`।

প্রশ্ন: AI summary যে ভুল সংখ্যা বলে না, তার code?

> **উত্তর:** `services/aiReportSummaryService.js`: `buildSummaryFacts()` শুধু aggregate বানায়; `allowedNumbers()` সেই facts-এর সব সংখ্যার set; `onlyKnownNumbers()` প্রতিটা sentence-এর সংখ্যা চেক করে, অজানা হলে বাদ (ছোট সংখ্যা ≤10 আর 24/30/90 context হিসেবে চলে)।

প্রশ্ন: Status কোথায় define করা? "parcel", "rider" কেন?

> **উত্তর:** `utils/repairRequestStatus.js`। প্রজেক্ট একটা parcel-delivery codebase থেকে শুরু হয়েছিল; data নষ্ট না করার জন্য stored value (`rider`, `parcel_picked_up`, `deliveryStatus`) রেখে দিয়েছি — এটা internal contract। User সবসময় repair শব্দ দেখে (Technician, Device collected)। পরিকল্পিত migration future work-এ আছে।

৫. সম্ভাব্য প্রশ্ন — General / Design

প্রশ্ন: এটা কি শুধু একটা CRUD app না?

> **উত্তর:** না। Repair একটা case যেটার 14টা state; প্রতিটা action-এ server role + current state চেক করে। টাকা আর assignment বদলানোর সব কাজ MongoDB transaction-এ, deadline আছে যাতে কিছু আটকে না থাকে, আর payment webhook-ভিত্তিক।

প্রশ্ন: MongoDB কেন, SQL কেন না?

> **উত্তর:** একটা repair request নিজের quote, inspection, history, settlement নিয়ে একটা document — document database-এ মানায়। আলাদা collection-এর মধ্যে consistency রাখতে MongoDB multi-document transaction (replica set) ব্যবহার করেছি। Relational হলেও করা যেত; এটা trade-off, ভুল-সঠিক না।

প্রশ্ন: Firebase Auth কেন? নিজে JWT বানালে না কেন?

> **উত্তর:** Password রাখা, email verification, Google login, token refresh — নিরাপদভাবে নিজে বানানো ঝুঁকিপূর্ণ। Firebase ID token server-এ Firebase Admin দিয়ে verify করি; role নিজের DB-তে।

প্রশ্ন: Serverless (Vercel) কেন? সমস্যা কী?

> **উত্তর:** Free, push করলেই deploy, scale নিজে থেকে। সমস্যা cold start — প্রথম request ধীর। API Mumbai region-এ সরিয়ে (DB-র পাশে) cold 10.7s → 2.9s, warm 0.75s → 0.2s হয়েছে। Rate limit memory-তে রাখা যায় না বলে MongoDB store।

প্রশ্ন: Frontend-এ state কীভাবে manage করেছ?

> **উত্তর:** Server data TanStack Query দিয়ে (cache, refetch, mutation-এর পর invalidate; key গুলো `hooks/*Keys.js`-এ)। Form-এ react-hook-form। Global client state কম — login Firebase context-এ।

প্রশ্ন: ER diagram বুঝিয়ে বলো।

> **উত্তর:** User অনেক repair request করে, technician-ও হতে পারে (technicians collection)। প্রতিটা request একটা service definition দিয়ে priced, অনেক job application (offer) পায়, তার payment rows, একটা review, সর্বোচ্চ ২টা warranty claim। Quote আর inspection request-এর ভেতরেই embedded। Withdrawal technician-এর সাথে।

প্রশ্ন: তুমি নিজে কী করেছ?

> **উত্তর:** পুরো project আমার: requirement, design, server আর client code, test, deploy, report। AI tool সাহায্য হিসেবে ব্যবহার করেছি, কিন্তু প্রতিটা feature-এর rule আর code আমি বুঝি — যেকোনো file খুলে বোঝাতে পারব।

৬. Feature gap (কী নেই, কেন নেই)

প্রশ্ন: Stripe তো বাংলাদেশে চলে না। তাহলে?

> **উত্তর:** ঠিক, তাই test mode-এ। Payment code একটা service-এর পিছনে (`paymentController` + `paymentProcessor`), তাই SSLCOMMERZ বা bKash Payment Gateway বসানো যাবে — future work-এ লেখা।

প্রশ্ন: bKash payout automatic না কেন?

> **উত্তর:** bKash disbursement API-র জন্য merchant account লাগে (খরচ ও approval)। তাই admin bKash Send Money দিয়ে পাঠায় আর TrxID record করে। Wallet হিসাব পুরোপুরি automatic আর exact।

প্রশ্ন: Technician-এর live location নেই কেন?

> **উত্তর:** Scope-এর বাইরে রেখেছি; এখন district/region দিয়ে matching। Future work-এ আছে।

প্রশ্ন: SMS notification নেই কেন? বাংলাদেশে তো SMS বেশি চলে।

> **উত্তর:** ঠিক, কিন্তু SMS gateway paid। এখন in-app + email (47 ধরনের notification)। Future work-এ SMS আছে।

প্রশ্ন: পুরো Bangla interface নেই কেন?

> **উত্তর:** AI helper বাংলা/Banglish ইনপুট বোঝে, offer note বাংলায় লিখতে পারে; কিন্তু পুরো UI localisation করিনি — future work।

প্রশ্ন: একজন technician একসাথে একটাই job কেন?

> **উত্তর:** Double booking আর pickup সংঘর্ষ আটকাতে। এটা limitation — ছোট অনেক job-এর জন্য daily capacity model future work।

প্রশ্ন: Cash payment নেই কেন?

> **উত্তর:** Cash দিলে platform-এর ১০% আর হিসাব রাখা কঠিন। Future work: hand-over-এ cash, তারপর wallet থেকে platform share কাটা।

প্রশ্ন: Load test বা security audit হয়েছে?

> **উত্তর:** না — limitation-এ সৎভাবে লেখা। Automated test আছে 2,761টা, কিন্তু real user আর load test হয়নি।

প্রশ্ন: Signed-in page-এর browser test নেই কেন?

> **উত্তর:** Sign-in-এর জন্য আসল Firebase account লাগে। Signed-in screen-এর পিছনের প্রতিটা endpoint server test-এ আছে, আর হাতে walkthrough করেছি (Chapter 7-এর screenshot)।

৭. Edge case প্রশ্ন

প্রশ্ন: Technician fee নিয়ে এলোই না (no-show)?

> **উত্তর:** Admin "missed pickup" থেকে withdraw করে (`POST /admin/repair-requests/:id/missed-pickup/withdraw`, `missedPickupController`) → fee পুরো refund, ওই offer `no_show`, job আবার open। No-show reliability score কমায়।

প্রশ্ন: Customer বাসায় ছিল না?

> **উত্তর:** Technician নোটসহ "Customer wasn't available" চাপে (`POST …/customer-not-available`, `workflowController.customerNotAvailable()`)। Request-এ `pickupRescheduleRequestedAt` বসে, customer-কে নতুন pickup সময় বেছে নিতে বলা হয়। Technician-ই না এলে সেটা আলাদা — admin-এর missed-pickup flow (`ask-new-time` বা withdraw)।

প্রশ্ন: একই সময়ে দুইজন customer slot-এর শেষ জায়গা নিলে?

> **উত্তর:** Capacity check হয়, কিন্তু lock নেই — একই সেকেন্ডে দুইজন পেতে পারে। এটা limitation-এ লেখা; এখনকার scale-এ ঠিক আছে, busy হলে atomic counter লাগবে।

প্রশ্ন: Payment দিয়ে দিল, কিন্তু ঠিক তখন অন্য offer বেছে নেওয়া হয়েছে / request cancel?

> **উত্তর:** Webhook-এ `bookApplication()` `JOB_NOT_OPEN` বা `TECHNICIAN_BUSY` ফেরত দেয় → কিছু book হয় না → fee নিজে থেকে refund। Cancel-এর পরে দেরিতে আসা payment-ও refund হয়।

প্রশ্ন: Stripe refund fail করলে?

> **উত্তর:** Refund "pending" থাকে, admin attention list-এ "Refund waiting" দেখায়, admin retry করে (`POST /admin/repair-requests/:id/inspection-fee/retry-refund`)। Idempotency key থাকায় retry-তে double refund হয় না।

প্রশ্ন: Customer quote approve করল কিন্তু টাকা দিল না?

> **উত্তর:** ৩ দিন পর cron বন্ধ করে দেয় (`closeUnpaidApprovedQuotes`)। Customer নিজেও cancel-unpaid করতে পারে।

প্রশ্ন: Customer quote decline করল, তারপর কিছুই করল না?

> **উত্তর:** Technician revise করে নতুন quote দিতে পারে; নয়তো ৩ দিন পর auto close (`declinedQuoteClose.js`), fee রাখা হয় (technician ৯০%)।

প্রশ্ন: Phone repair করা যাবে না?

> **উত্তর:** Inspection-এ "Not repairable / Not economical" → `POST …/close-not-repairable`; fee রাখা হয় কারণ technician পরিদর্শন করেছে।

প্রশ্ন: Repair শুরু করে technician শেষ করতে পারল না?

> **উত্তর:** "Can't finish this repair" (`POST …/repair/cannot-complete`) → admin-এর কাছে যায় → admin cancel-paid-repair দিয়ে refund করে, device ফেরতের confirm আলাদা step।

প্রশ্ন: Customer বলছে phone ফেরত পায়নি?

> **উত্তর:** "I haven't received it" → `receipt-dispute`, admin সিদ্ধান্ত দেয়। ততক্ষণ technician-এর টাকা pending থাকে।

প্রশ্ন: Customer confirm-ই করে না?

> **উত্তর:** Technician "returned" mark করার ৩ দিন পর auto confirm (`receiptConfirmation.js`, method `'auto'`)।

প্রশ্ন: Warranty-র অপব্যবহার — নতুন করে ফেলে দিয়ে claim করলে?

> **উত্তর:** Covered শুধু একই fault ফিরে এলে বা লাগানো part নষ্ট হলে। Technician free re-inspection-এ দেখে "Not covered" (new damage, water, drop) দিতে পারে কারণসহ; customer admin review চাইতে পারে। Pickup condition record এখানে প্রমাণ।

প্রশ্ন: Technician warranty claim ignore করলে?

> **উত্তর:** প্রতিটা step-এর deadline (book 3 দিন, decide 3 দিন, re-repair 7 দিন)। মিস হলে overdue → admin take over → দরকারে refund।

প্রশ্ন: Warranty refund-এর আগে technician টাকা তুলে নিলে?

> **উত্তর:** Settlement `reversed` হয়; আগে তোলা টাকা "owed" হিসেবে দেখায় আর পরের আয় থেকে কাটা হয় (`calculateWallet`)।

প্রশ্ন: Admin technician remove করল, কিন্তু তার wallet-এ টাকা আছে?

> **উত্তর:** Remove-এর সময় একই transaction-এ পুরো available balance bKash withdrawal হিসেবে queue হয় (`services/technicianRemovalPayout.js`); আগে preview দেখায় (`GET /technicians/:id/final-payout`)। Open warranty claim থাকলে remove করা যায় না।

প্রশ্ন: Request চলাকালীন admin দাম বদলালে?

> **উত্তর:** পুরনো request-এ price snapshot থাকে; নতুন দাম শুধু নতুন request-এ (`pricingRule.version`)।

প্রশ্ন: Region-এ কোনো technician নেই?

> **উত্তর:** Request নেওয়াই হয় না (409, কারণসহ), customer অপেক্ষায় পড়ে থাকে না; turned-away count admin report-এ দেখায় — কোথায় technician লাগবে।

প্রশ্ন: AI বন্ধ / ধীর / quota শেষ?

> **উত্তর:** সব AI optional। Timeout আছে (`aiHelperCall.js`), support chat-এ fallback model; বন্ধ হলে form হাতে পূরণ করা যায়, বাকি app স্বাভাবিক।

প্রশ্ন: Webhook দেরিতে এলো, customer page reload করল?

> **উত্তর:** Page শুধু দেখায়; webhook এলে state বদলায়। দুইবার এলেও একবার record।

প্রশ্ন: Customer দুইবার checkout খুললে?

> **উত্তর:** `services/checkoutSessionManager.js` একটা request-এ একটাই খোলা checkout রাখে (claim + TTL), Stripe create-এ idempotency key।

৮. Security প্রশ্ন

প্রশ্ন: কেউ অন্যের request id দিয়ে data দেখতে পারবে?

> **উত্তর:** না। Controller দেখে caller owner, assigned technician বা admin কি না; নাহলে 403/404। Public tracking-এ কোনো personal data নেই।

প্রশ্ন: Tracking code অনুমান করা যাবে?

> **উত্তর:** 128-bit random (`crypto.randomBytes(16)`) — কার্যত অসম্ভব। আর দেখালেও শুধু stage দেখায়।

প্রশ্ন: Photo-র link share করলে?

> **উত্তর:** Bucket private; link কয়েক মিনিটের signed URL, তারপর অকেজো। Upload-এর পর আসল type ও size আবার verify করা হয়।

প্রশ্ন: NoSQL injection?

> **উত্তর:** Input-এর type/shape validate করা হয় (`utils/*` validator), search text escape (`utils/searchSanitize.js`), ObjectId parse-এর আগে চেক।

প্রশ্ন: Secret কোথায়?

> **উত্তর:** শুধু server environment variable-এ (Vercel); repo বা browser-এ না। `.env.example`-এ শুধু নাম। Log-এ request body লেখা হয় না (`utils/safeLogger.js`)।

প্রশ্ন: CORS আর rate limit?

> **উত্তর:** `config/cors.js`-এ allow-list (production-এ শুধু Vercel client)। AI chat-এ per-IP/per-user আর site-wide limit, MongoDB-তে রাখা।

প্রশ্ন: Cron URL কেউ নিজে call করলে?

> **উত্তর:** Secret (`CRON_SECRET`) না মিললে কিছু চলে না; secret ছাড়া configure না থাকলে 503।

প্রশ্ন: Email verify না করে কেউ request বা payment করতে পারবে?

> **উত্তর:** না — `verifyEmailVerified` middleware 403 দেয় (create request, payment, review, warranty claim-এ)।

৯. AI প্রশ্ন

প্রশ্ন: AI কি দাম বানিয়ে বলতে পারে?

> **উত্তর:** না। কোনো AI feature দাম সেট করে না। Support chat-এর দাম live catalogue থেকে prompt-এ যায়; request helper শুধু catalogue id বেছে নেয়, server আবার চেক করে।

প্রশ্ন: Customer-এর personal data AI-তে যায়?

> **উত্তর:** নাম, email, card কিছু পাঠানো হয় না। Request helper-এ শুধু customer-এর লেখা আর (দিলে) photo — form-এ সতর্কবার্তা আছে। Report summary শুধু aggregate সংখ্যা।

প্রশ্ন: AI ভুল উত্তর দিলে?

> **উত্তর:** Output schema validate হয়; ভুল হলে বাদ। Helper-এর সব suggestion customer বদলাতে পারে, Undo আছে। AI ছাড়াই সব কাজ করা যায়।

প্রশ্ন: AI কত দ্রুত? আগে fail করত কেন?

> **উত্তর:** আগের model ধীর ছিল, timeout হতো। এখন `gemini-3.5-flash-lite`, fail করলে fallback model; live-এ 1.4–1.9s।

১০. Testing আর deployment

প্রশ্ন: কীভাবে test করেছ?

> **উত্তর:** 2,761 server test, real HTTP দিয়ে, real MongoDB replica set-এ (transaction চালাতে), Stripe/AI/storage fake। Race test-ও আছে (দুই admin একসাথে assign)। Client-এ 58 Playwright browser test (desktop + mobile)। GitHub Actions প্রতিটা push-এ; সব pass হলে তবেই main-এ push।

প্রশ্ন: Test DB আর production DB আলাদা?

> **উত্তর:** হ্যাঁ। Test throwaway/in-memory MongoDB-তে চলে, কখনো আসল credential দিয়ে না। Production server secret আর https origin ছাড়া start-ই হয় না।

প্রশ্ন: Migration কীভাবে করেছ (যেমন minimum fee)?

> **উত্তর:** Script আগে dry-run (কী বদলাবে দেখায়), তারপর confirm flag দিয়ে চালানো; production guard আছে। Minimum fee migration production আর local দুটোতেই চালানো হয়েছে।

১১. কঠিন / ব্যক্তিগত প্রশ্ন

প্রশ্ন: সবচেয়ে কঠিন অংশ কী ছিল?

> **উত্তর:** টাকা আর state একসাথে ঠিক রাখা — payment, booking, refund, settlement যেন কখনো অর্ধেক না হয়। এর জন্য transaction, webhook-as-truth আর idempotency key। আর "কোনো request আটকে থাকবে না" — প্রতিটা waiting state-এ deadline বা admin alert।

প্রশ্ন: আবার শুরু করলে কী বদলাতে?

> **উত্তর:** শুরু থেকেই repair domain-এর নাম (parcel/rider legacy না রেখে), local payment gateway, আর slot capacity-র জন্য atomic counter।

প্রশ্ন: এটা কি আসলে চালু করা যাবে?

> **উত্তর:** Technically deploy করা আছে, live। বাস্তবে চালুর আগে লাগবে: local payment gateway, আসল দাম (repair shop-এর সাথে ঠিক করা), technician verification প্রক্রিয়া, আর load/security test।

প্রশ্ন: Business model কী?

> **উত্তর:** প্রতিটা repair-এ ১০% commission (kept inspection fee-তেও একই ভাগ)। Customer বাড়তি কিছু দেয় না।

প্রশ্ন: অন্য platform থেকে আলাদা কীভাবে?

> **উত্তর:** Inspection-এর পর itemised quote, approve-এর আগে চার্জ নেই, condition record, 30-দিনের warranty workflow, region-ভিত্তিক matching আর ranking, আর technician-এর guaranteed payment (আগে online payment, receipt confirm-এ release)।

শেষ টিপস: Demo-র আগে `RepairRequestV2Form.jsx`, `jobBooking.js`, `settlement.js`, `offerRanking.js`, `paymentController.js` (webhook অংশ) আর `middleware/auth.js` একবার খুলে দেখে নাও — "code দেখাও" বললে এগুলোই সবচেয়ে বেশি চাওয়া হয়। VS Code-এ **Ctrl+P** দিয়ে file নাম লিখে দ্রুত খোলো, **Ctrl+Shift+F** দিয়ে route path (যেমন `/applications`) খুঁজে দেখাও।